10.6 Arrays and Pointers as Function Arguments

[This section corresponds to K&R Sec. 5.2]

Earlier, we learned that functions in C receive copies of their arguments. (This means that C uses *call by value*; it means that a function can modify one of its arguments without modifying the value in the caller.) We didn't say so at the time, but when a function is called, the copies of the arguments are made as if by assignment. But since arrays can't be assigned, how can a function receive an array as an argument? The answer will explain why arrays are an apparent exception to the rule that functions cannot modify their arguments.

We've been regularly calling a function `getline` like this:

```
char line[100];
getline(line, 100);
```

with the intention that `getline` read the next line of input into the character array `line`. But in the previous paragraph, we learned that when we mention the name of an array in an expression, the compiler generates a pointer to its first element. So the call above is as if we had written

```
char line[100];
getline(&line[0], 100);
```

In other words, the `getline` function does *not* receive an array of `char` at all; it actually receives a pointer to `char`!

As we've seen throughout this chapter, it's straightforward to manipulate the elements of an array using pointers, so there's no particular insurmountable difficulty if `getline` receives a pointer. One question remains, though: we had been defining `getline` with its `line` parameter declared as an array:

```
int getline(char line[], int max)
{
...
}
```

We mentioned that we didn't have to specify a size for the `line` parameter, with the explanation that `getline` really used the array in its caller, where the actual size was specified. But that declaration certainly does look like an array--how can it work when `getline` actually receives a pointer?

The answer is that the C compiler does a little something behind your back. It knows that whenever you mention an array name in an expression, it (the compiler) generates a pointer to the array's first element. Therefore, it knows that a function can never actually receive an array as a parameter. Therefore, whenever it sees you defining a function that seems to accept an array as a parameter, the compiler quietly pretends that you had declared it as accepting a pointer, instead. The definition of `getline` above is compiled exactly as if it had been written

```
int getline(char *line, int max)
{
...
}
```

Let's look at how `getline` might be written if we thought of its first parameter (argument) as a pointer, instead:

```
int getline(char *line, int max)
{
int nch = 0;
int c;
max = max - 1;                  /* leave room for '\0' */

#ifndef FGETLINE
while((c = getchar()) != EOF)
#else
while((c = getc(fp)) != EOF)
#endif
        {
        if(c == '\n')
                break;

        if(nch < max)
                {
                *(line + nch) = c;
                nch = nch + 1;
                }
        }

if(c == EOF && nch == 0)
        return EOF;

*(line + nch) = '\0';
return nch;
}
```

But, as we've learned, we can also use ``array subscript'' notation with pointers, so we could rewrite the pointer version of `getline` like this:

```
int getline(char *line, int max)
{
int nch = 0;
int c;
max = max - 1;                  /* leave room for '\0' */

#ifndef FGETLINE
while((c = getchar()) != EOF)
#else
while((c = getc(fp)) != EOF)
#endif
        {
        if(c == '\n')
                break;

        if(nch < max)
                {
                line[nch] = c;
                nch = nch + 1;
                }
        }

if(c == EOF && nch == 0)
        return EOF;

line[nch] = '\0';
return nch;
}
```

But this is exactly what we'd written before (see chapter 6, Sec. 6.3), except that the declaration of the `line` parameter is different. In other words, within the body of the function, it hardly matters

whether we thought `line` was an array or a pointer, since we can use array subscripting notation with both arrays and pointers.

These games that the compiler is playing with arrays and pointers may seem bewildering at first, and it may seem faintly miraculous that everything comes out in the wash when you declare a function like `getline` that seems to accept an array. The equivalence in C between arrays and pointers can be confusing, but it *does* work and is one of the central features of C. If the games which the compiler plays (pretending that you declared a parameter as a pointer when you thought you declared it as an array) bother you, you can do two things:

1. Continue to pretend that functions can receive arrays as parameters; declare and use them that way, but remember that unlike other arguments, a function can modify the copy in its caller of an argument that (seems to be) an array.
2. Realize that arrays are always passed to functions as pointers, and always declare your functions as accepting pointers.

Read sequentially: prev next up top